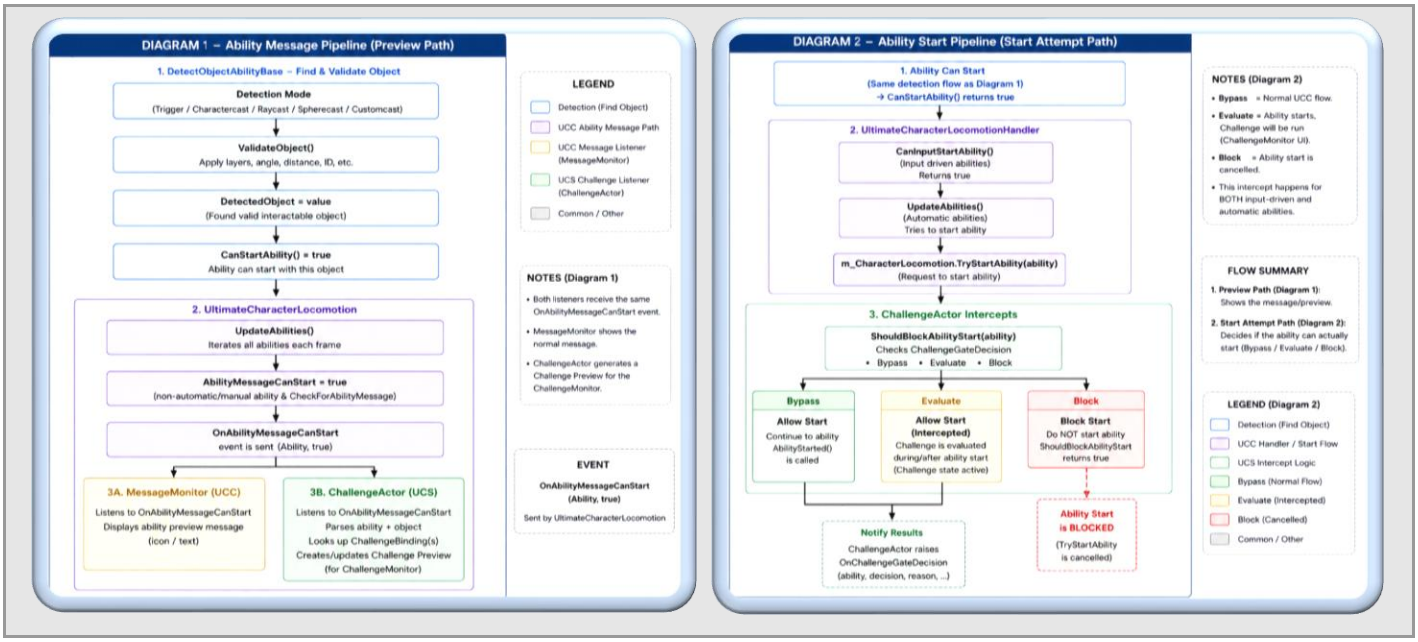
Ultimate Challenge System for Opsive Character Controllers

Table of Contents

Getting Started

The Ultimate Challenge System is an add-on framework for Opsive Character Controllers. It allows ordinary UCC abilities such as Interact or other DetectObjectAbilityBase-derived abilities to be intercepted, previewed, gated, and conditionally resumed through configurable Challenge assets.

- **Use it for:** locked doors, keycards, terminals, skill checks, inventory gates, faction locks, password panels, lockpicking, timed minigames, and any interaction that should require a condition before the original ability runs.
- **Do not use it for:** simple interactions that never need requirements, UI previews, retries, cooldowns, persistence, or custom gameplay.
- **Core idea:** the player still presses the same interaction input. UCS quietly sits in front of the original ability and decides what should happen.

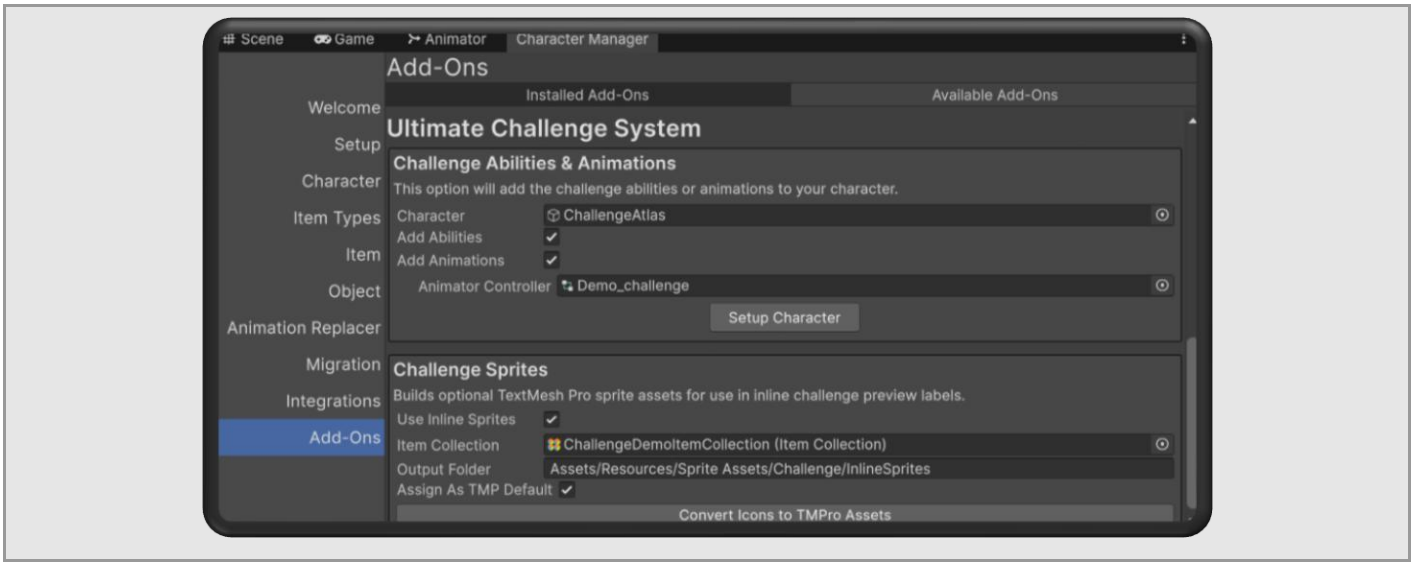


Basic Requirements

- An Opsive character controller must already be imported (UCC, UFPS, TPC)
- Your scene should contain the standard Opsive game managers and UI components.
- TextMeshPro is optional but recommended for inline item icons in preview text.
- ObjectIdentifier IDs should be used consistently on interactable objects.

Recommended Setup Order

1. Import the desired Opsive character controller.
2. Import the Ultimate Challenge System package.
3. Open the Opsive Add-Ons Manager: Tools → Opsive → Add-Ons Manager. Navigate to Installed Add-Ons → Ultimate Challenge System.
4. Assign an existing Opsive character.
5. Use the manager to add ChallengeActor and required animation setup.
6. Create or assign a UI canvas with ChallengeMonitor.
7. Create at least one Challenge asset.
8. Assign that Challenge asset to a ChallengeActor binding for the ability you want to intercept.
9. Place an ObjectIdentifier on the target world object and make sure its ID matches the Challenge asset TargetID.
10. Enter Play Mode and test the preview before building complex interactive behavior.



Example Setup: Locked Door

This example demonstrates the complete setup flow for a basic locked door interaction using an Inventory Challenge.

By the end of this setup:
- The player can approach a door.
- A preview prompt appears.
- On Interact, the challenge checks for a required key item.
- The door interaction succeeds only if the player has the item.

Setup

Step 1 — Add an ObjectIdentifier to the Door

- Select the door GameObject in the scene.
- Add an ObjectIdentifier component.
- Set the ObjectIdentifier ID to a unique value such as 11001.
- IMPORTANT: The Challenge asset Target ID must match this ObjectIdentifier ID exactly.

Step 2 — Verify the Character Has an Interact Ability

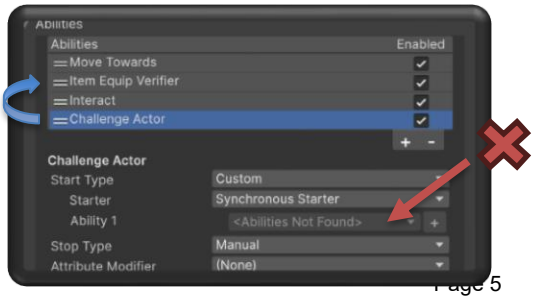
- The character must already contain an ability derived from DetectObjectAbilityBase.
- Most commonly this is Interact.
- Test the Interact ability before adding challenges.
- If the character cannot already interact with the door normally, fix that first.

Step 3 — Add ChallengeActor

- Select the player character.
- Add ChallengeActor to their Ability list.
- Recommended Settings:
- Start Type = Custom
- Stop Type = Manual
- IMPORTANT: ChallengeActor should sit above the intercepted abilities in the ability list.

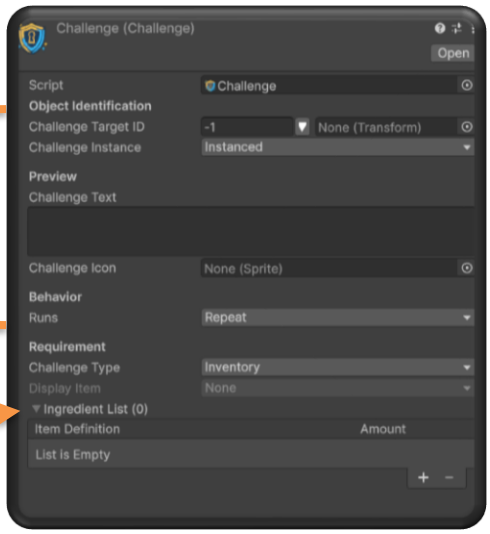
Step 4 — Configure SynchronousStarter

- Assign SynchronousStarter as the Ability Starter.
- Add the Interact ability to the intercepted abilities list. If you do not see the Interact ability inside of the intercept list, then the owning ability needs to be reordered above it.
- This allows ChallengeActor to mirror the Interact ability's startup conditions automatically.



Step 5 — Create the Challenge Asset

- Create a new Challenge asset from:
- Assets → Create → Opsive → Ultimate Challenge System → Challenge



Configure the Challenge Asset

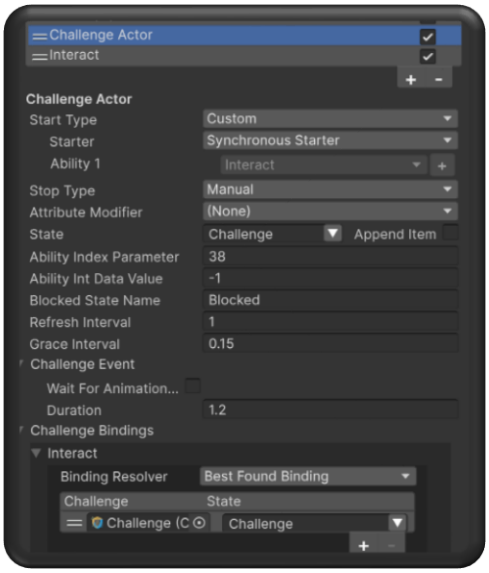
- Target ID = 11001
- Challenge Instance = Instanced
- Challenge Text = “This door is locked…”
- Challenge Icon = Choose an accompanying sprite asset
- Runs = OnSuccess
- Type = Inventory

Configure Inventory Requirements

- Add a required inventory ItemType – can be anything in your character’s ItemCollection.
- Required amount should be 1.
- Mark the required ItemType as Consumable if the item should be removed from inventory after the challenge concludes.

Step 6 — Bind the Challenge

- Return to the player character.
- Inside ChallengeActor’s Challenge Bindings section, locate the “Interact” challenge group.
- Add a new Challenge Binding.
  - Assign the newly created challenge asset.
  - Optional: Assign a State such as LockedDoor for character animations / audio / feedbacks



Step 7 — Add ChallengeMonitor Prefab

- Create or select a UI Canvas.
- Inside of the Challenge Demo files, assign the ChallengeMonitor Prefab underneath the new Canvas.
  - Assets → Opsive → UltimateCharacterController → Add-Ons → Challenge → Demo → Prefabs → UI_Prefabs

Test the Scene

- Approach the door - the Preview message should display your configured preview text / icon.
- With the keycard, the challenge passes and the door interaction succeeds.
- Future interactions will use the default MessageMonitor path

Troubleshooting

No Preview Appears

- Verify ChallengeMonitor exists.
- Verify ChallengePreviewObject exists.
- Verify the door contains ObjectIdentifier.
- Verify the Target ID matches the Challenge asset.
- Verify the Interact Ability has its AbilityMessageText and/or its AbilityMessageIcon assigned in the inspector.

The Door Always Opens

- Verify ChallengeActor is above Interact in the ability list.
- Verify SynchronousStarter intercepts Interact.

- Verify the Challenge is bound to the correct ability group.

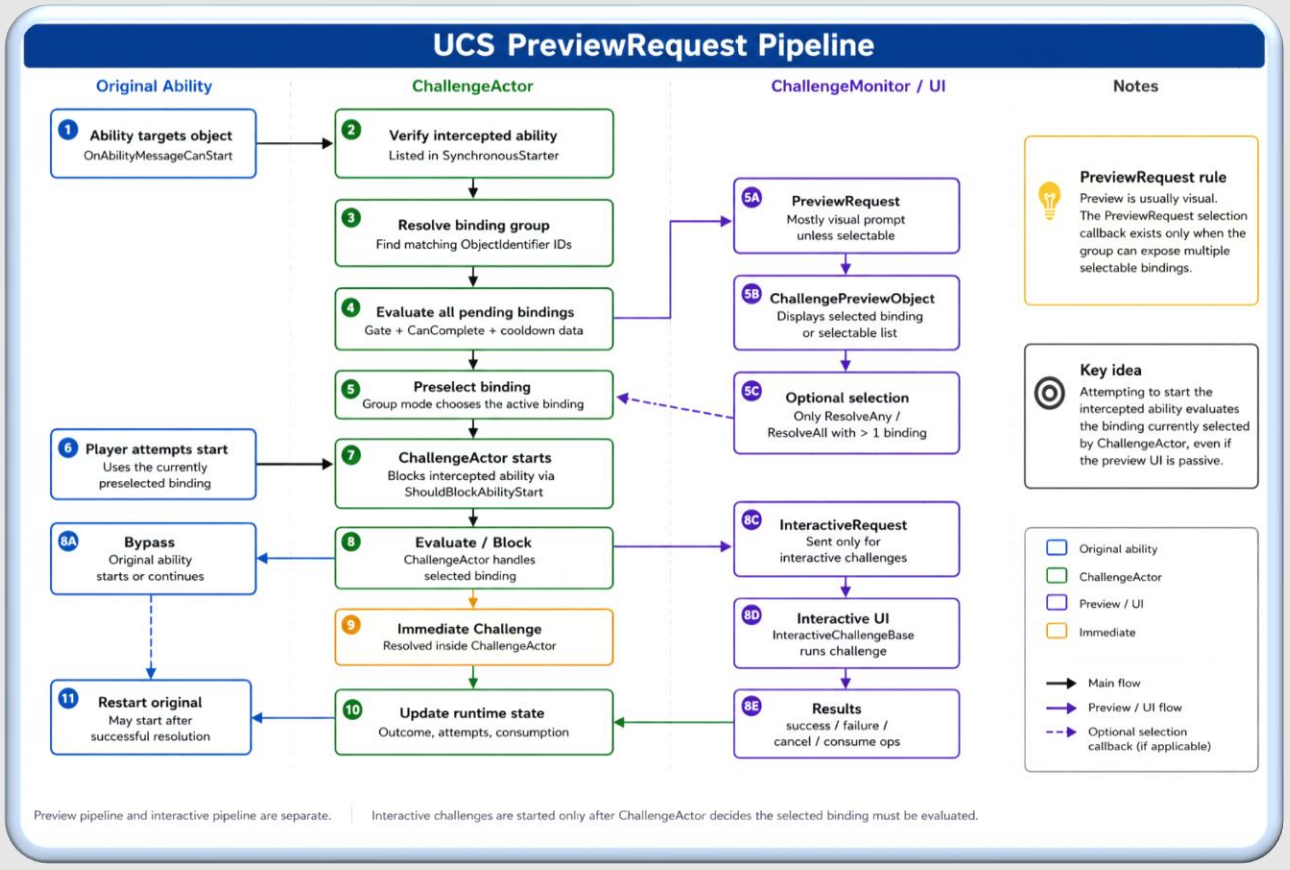
The Challenge Never Passes

- Verify the inventory item identifier matches exactly.
- Verify the required amount is correct.
- Verify the inventory system contains the item at runtime.

Runtime Flow Overview

UCS is easiest to understand as something of a middleman; every valid character ability can be seamlessly intercepted using conditional behavior described by Challenge assets. The ChallengeActor ability will listen for attempts to start an ability, evaluate the ability's detected object, and either allow the ability to continue, block it, or evaluate a configured Challenge asset.

1. An ability triggers the OnAbilityMessageCanStart event while looking at or targeting an object.
2. ChallengeActor verifies that the ability is listed in SynchronousStarter.
3. ChallengeActor resolves all ChallengeBindings for that ability and attempts to find any matching ObjectIdentifier IDs on the object.
4. If the found binding is not bypassed, an OnChallengePreviewRequest is sent.
   - ChallengeMonitor receives the PreviewRequest and displays a ChallengePreviewObject.
   - Some group modes may also expose an optional selection callback for multi-binding previews.
5. If the selected binding must evaluate, ChallengeActor starts and blocks the intercepted ability.
6. Immediate challenges resolve inside ChallengeActor.
7. Interactive challenges send an InteractiveRequest to ChallengeMonitor.
8. ChallengeMonitor starts the matching InteractiveChallengeBase UI or gameplay controller.
9. The interactive challenge returns success, failure, cancellation, and consumption operations.
10. ChallengeActor updates runtime state and may restart the intercepted ability.



NOTE: The PreviewRequest and InteractiveRequest pipelines are separate events sent by the ChallengeActor Ability.

- Previews tell the player what challenges are detected on the object and their requirements.
- Interactive challenge requests are only sent when the selected challenge binding is Interactive; the selected Interactive challenge must then be evaluated (minigame starts and then must be resolved).

Runtime Components:	
Component	Description
ChallengeActor	Concurrent character ability that intercepts configured abilities, resolves bindings, builds previews, evaluates conditions, and applies results.
SynchronousStarter	AbilityStarter that mirrors the start conditions of selected abilities so ChallengeActor can start at the correct time.
ChallengeObject	Optional persistence bridge for world objects that should remember challenge outcomes through save/load.
ChallengeMonitor	CharacterMonitor responsible for preview UI, interactive challenge UI, input locking, cursor behavior, and result routing.
ChallengePreviewObject	UI renderer for passive prompts, single binding previews, multi-binding selectors, blocked states, and world-follow previews.
InteractiveChallengeBase	Base MonoBehaviour for interactive challenge UIs and minigames.

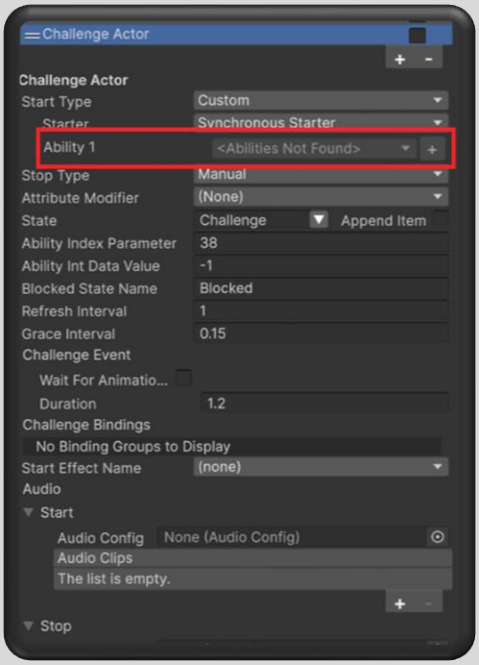
ChallengeActor Ability

ChallengeActor is the character-side controller for UCS. It should be added to the same character that owns the ability being intercepted. It is configured as a concurrent ability with a custom starter, and it always sits above the abilities that it gates.

NOTICE: If ChallengeActor's StartType is ever misconfigured or the SynchronousStarter doesn't have any intercepted abilities, on PlayMode start the ChallengeActor will be safely disabled.

Ability Settings

Field	What It Does	Recommended Use / Notes
Start Type	Must be Custom.	ChallengeActor starts through SynchronousStarter, not through its own input.
Ability Starter	Must be SynchronousStarter.	This is what lets the ability mirror the start conditions of other abilities.
Stop Type	Must be Manual.	ChallengeActor controls when the challenge attempt ends.
Concurrent	Always true.	It must be able to run alongside or in front of normal character behavior.
Ability Int Data Value	Value sent to the animator through AbilityIntData.	Use when the challenge animation layer needs to know which challenge animation to play.
Blocked State Name	State sent to the character and any state-aware DetectedObjects when a blocked challenge is attempted.	Useful for locked buzzers, blocked character / target animations, red lights, or failed interaction feedbacks (such as audio).
Refresh Interval	How often preview elements refresh while hovered.	Primarily used to update the Preview text / icon to indicate challenge state such as cooldown timers.
Grace Interval	Short delay before a preview closes after an automatic target is lost.	Prevents excessive target loss and re-acquisition when moving in and out of detection range.
Challenge Event	Animation event used to signal the ChallengeActor to stop precisely when intended after starting.	Use animation events for physical challenge animations; use timed fallback for simple cases.

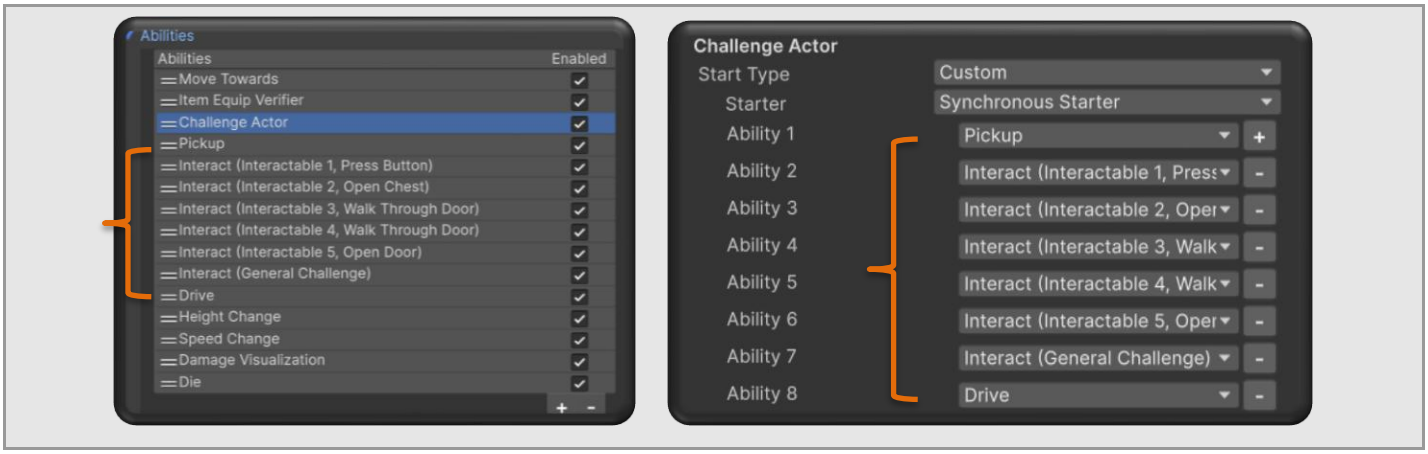


SynchronousStarter

SynchronousStarter is a Custom AbilityStarter that watches selected abilities and will start the owning ability whenever any of these abilities attempt to start. This entirely removes the need for custom input or other conditional startup behavior for your owning ability.

The SynchronousStarter can be used on any Ability you wish to start in conjunction with another ability starting – for example, your character will Jump every time they start Sprinting.

Field	What It Does	Recommended Use / Notes
Intercept Ability IDs	Serialized ability indices discovered from the character ability list.	Used for fast runtime lookup but can become stale when ability order changes.
Intercepted Abilities	Runtime resolved ability references.	These are the abilities ChallengeActor is allowed to preview and gate.



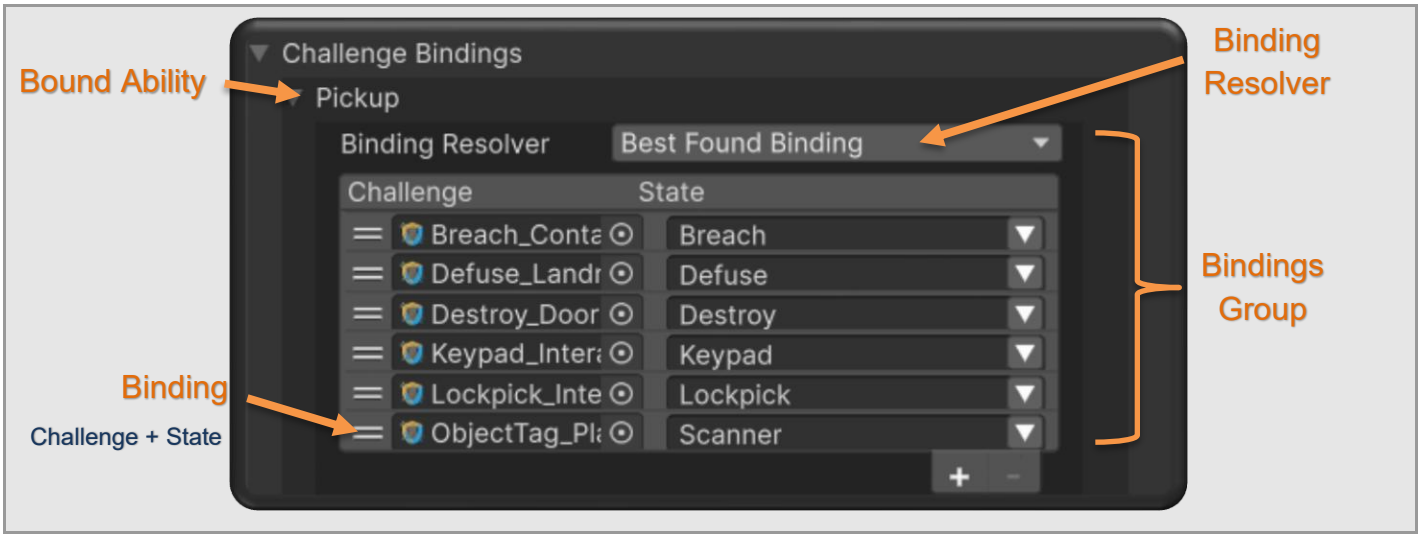
IMPORTANT: The SynchronousStarter's inspector code should be refreshed each time an Ability is added, removed, or reordered – this is because Opsive's Abilities are currently only trackable via their index in the Abilities array, thus anytime a change is made the data must be updated in order to keep a live reference.

Challenge Binding Groups

ChallengeActor stores challenge bindings per intercepted ability found in the SynchronousStarter. Each intercepted ability can have its own group of ChallengeBindings and its own binding resolution mode. This allows Interact, Drive, Pickup, or any other DetectObjectAbilityBase abilities to be gated differently on the same character.

NOTICE: Only abilities that inherit from DetectObjectAbilityBase can be assigned Challenges – every other ability type will be given a warning message and disallowed from adding challenge assets.

Field	What It Does	Recommended Use / Notes
Bound Ability	The ability this binding group belongs to.	Any DetectObjectAbilityBase-derived ability.
Challenge Binding Group	Array of Challenge + State pairs belonging to a BoundAbility	Each entry in the group points to one Challenge asset and an optional state name.
Binding Resolver	Controls how multiple bindings on the same detected object are resolved.	Choose carefully when one object may contain multiple challenge IDs or multiple available interaction options.
Challenge	The Challenge asset evaluated for the detected target.	The target ObjectIdentifier ID must match the Challenge Target ID.
State	Optional Opsive State activated during preview/evaluation.	Use this for character and valid state-enabled target presentation. Used to alter character animations, start / stop audio, effects, etc.



Binding Resolver Modes

Option	Meaning
BestFoundBinding	Presents a single binding; prefers bypassed results, then evaluatable results, then fallback entries. Best general-purpose mode when one object has several possible bindings.
FirstFoundBinding	Presents a single binding; selects the first valid binding found. Best when binding order is intentional and only one should matter.
NextFoundBinding	Presents a single binding; attempts one binding at a time and then advances to additional unresolved bindings. Useful for serial multi-step interactions that cannot be selected or performed out of order.
ResolveAnyBinding	Presents a list of all found bindings – these bindings are selectable via UI Buttons. Only one challenge in the group must be resolved; useful for alternate requirements, such as keycard OR hacking skill.
ResolveAllBindings	Presents a list of all found bindings – these bindings are selectable via UI Buttons. All found challenges in the group must be resolved before the intercepted ability can start.

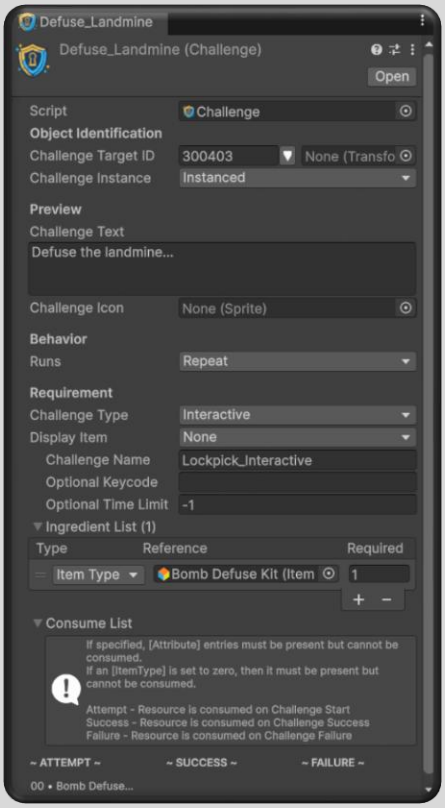
PRO TIP: NextFoundBinding and ResolveAllBindings are the most demanding modes. Use them when every found challenge on the DetectedObject must be solved.

Challenge Assets

Summary

A Challenge asset is the authoring definition for one challenge rule. It contains target lookup information, runtime memory behavior, run/reset rules, messages, type-specific payloads, and optional interactive challenge metadata.

Field	What It Does	Recommended Use / Notes
Target ID	IDObject pointing to the ObjectIdentifier ID associated with the world object.	The ID must exist somewhere in the detected object hierarchy. This is how the binding finds the subject.
Challenge Instance	Determines how runtime how state is remembered.	Choose Instanced, Persistent, or Global based on whether state belongs to a single object, saved object, or shared ID.
Challenge Message Text	Text shown by the preview UI when this challenge is active.	Use clear player-facing text. Supports placeholder formatting for requirements.
Challenge Message Icon	Sprite shown with the challenge preview.	Use icons for keys, batteries, terminals, locks, or challenge type.
Runs	Determines whether the challenge repeats, bypasses after success, or uses reset attempts.	OnSuccess is best for permanent unlocks. Repeat is best for requirements checked every interaction.
Reset	Determines how OnReset challenges become available again.	Only meaningful when Runs is OnReset.
Type	Selects the active payload.	ObjectTag, Inventory, SkillCheck, or Interactive.
Display Item Index	Optional index for forced display/equip item behavior.	Use when the character should visually present a required item during the challenge.
Reset Attempts	Number of failed attempts allowed for OnReset challenges.	Use for lockouts or limited-entry minigames.
Reset Cooldown	Cooldown duration before OnCooldown challenges reset.	Use for temporary lockouts or failed retry delay.
Time Limit	Optional time limit for interactive challenges.	Use -1 or <= 0 for no time limit.
Challenge Name	Name used by ChallengeMonitor to match an InteractiveChallengeBase UI.	Should match the target UI GameObject name by default.



PRO TIP: Challenge assets should be reusable where possible, but not at the cost of clarity. If two doors have different messages, different reset behavior, or different UI, create separate Challenge assets.

Settings

Instance Modes

Each Challenge asset tells the ChallengeActor how it should be treated in memory. Depending on the configuration of the challenge, solved doors, failed terminals, cooldowns, and attempts can be treated as separate entities or as a shared global state between all challenge objects.

Option	Meaning
Instanced	Use for one-off scene objects that do not need save/load state. The solved or failed result belongs to the resolved Transform instance in that scene.
Persistent	Use for objects whose state must survive save/load. Pair this with ChallengeObject on the target object.
Global	Use for shared locks or global game state. Every object with the same ID shares the same result.

Run Modes

Option	Meaning
Repeat	The challenge evaluates every time. Best for recurring item checks, temporary permissions, and interactions that should never become permanently solved.
OnSuccess	After success, future interactions bypass the challenge. Best for locked doors, unlocked chests, or one-time terminal access.
OnReset	The challenge tracks failures, attempts, cooldowns, and reset behavior. Best for lockouts and retry-limited minigames.

Reset Modes

Option	Meaning
OnCooldown	The challenge resets after ResetCooldown seconds. Failed lockouts and successful temporary bypass windows can both use cooldown behavior.
OnSceneLoad	The challenge resets when the scene reloads or when your runtime restores fresh scene state.
Never	The challenge does not automatically reset. Persistent challenges may remain solved or failed until changed by your own logic.

IMPORTANT: Reset rules are only meaningful for OnReset challenges. If Runs is set to either Repeat or OnSuccess, do not expect ResetCooldown to behave like a general timer unless the code path actually uses reset state.

Payloads

Challenges are divided into two broad categories.

1. **Immediate challenges:** Inventory, SkillCheck, and ObjectTag checks resolve directly inside ChallengeActor. No custom UI is opened.
2. **Interactive challenges:** Interactive challenge entry requirements are checked first, then ChallengeActor sends an InteractiveRequest to the ChallengeMonitor for asynchronous resolution.

NOTE: Use immediate challenges for requirements that can be answered instantly. Use interactive challenges when a minigame is more appropriate.

ObjectTag Challenge

ObjectTag challenges validate the character or actor context against one or more tags. They are useful for faction checks, AI-only interactions, player-only interactions, scripted proxies, or role-based access.

Field	What It Does	Recommended Use / Notes
Required Tags	Array of accepted tags.	If any required tag matches the character-side context, the challenge passes.

NOTE: By default the actor context checks the character GameObject tag. If your project needs faction components, network authority, or possession state, extend the context logic instead of overloading Unity tags.

Inventory Challenge

Inventory challenges require exact item identifiers and amounts. They are best for keycards, batteries, keys, consumable items, repair parts, or any interaction gated by inventory content.

Field	What It Does	Recommended Use / Notes
Items	ItemIdentifierAmount entries required to pass.	Each row should reference the exact item identifier and required amount.
Consumed Items	Indices of Items consumed on success.	Only include rows that should be removed. Leave empty for reusable keys.

PRO TIP: Do not consume reusable keys. A keycard that unlocks many doors should usually be required but not consumed.

SkillCheck Challenge

SkillCheck challenges allow mixed attribute and item requirements. They are useful when an interaction can depend on character stats, tools, or both.

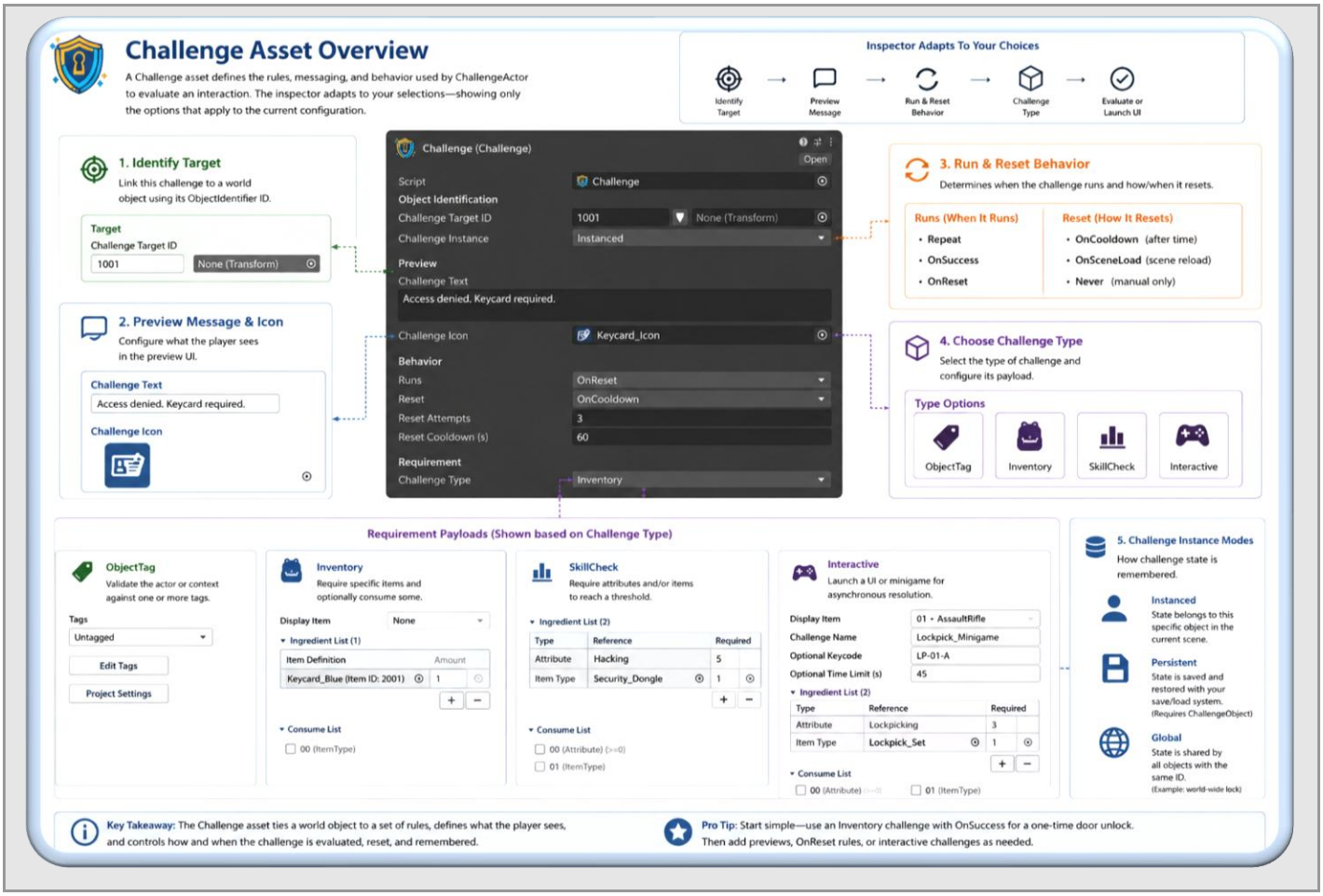
Field	What It Does	Recommended Use / Notes
Required Items Or Stats	Array of IngredientAmount rows.	Rows can represent an Attribute or ItemType.
Consumed Items Or Stats	Indices of item rows to consume.	Attribute rows act as checks and should not be consumed.

Interactive Challenge

Interactive challenges are validated for entry and then routed to ChallengeMonitor. They are best for password panels, hacking, lockpicking, timing challenges, alignment puzzles, or any gameplay interaction that should happen inside a UI/minigame.

Field	What It Does	Recommended Use / Notes
Ingredients	Items or attributes required to start the interactive attempt.	These are entry requirements and are not automatically consumed unless referenced by consume arrays.
Attempt Consumes	Ingredient indices consumed when an attempt starts or resets.	ItemTypes are spent when an interactive attempt begins or resets. Use for tokens, lockpicks, hacking charges, or energy spent per attempt.
Success Consumes	Ingredient indices consumed only on success.	ItemTypes are spent only after the challenge succeeds. Use for items inserted into the target, such as batteries or fuses.
Failure Consumes	Ingredient indices consumed only on failure.	ItemTypes are spent only after the challenge fails. Use for broken lockpicks, damaged tools, or penalty resources.
Expected Code	Optional password/code for challenges that need to match a specific sequence or pattern.	Use for Password-style challenges or custom UIs that read Challenge.InteractiveExpectedCode.

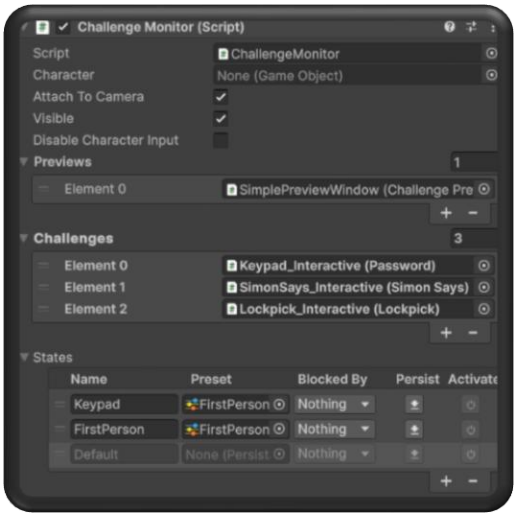
Conclusion



IMPORTANT: ItemTypes and Attributes can be required to fulfill a challenge without being consumed. This is how reusable keycards, certifications, class roles, and permanent tools should be authored.

Challenge Monitor

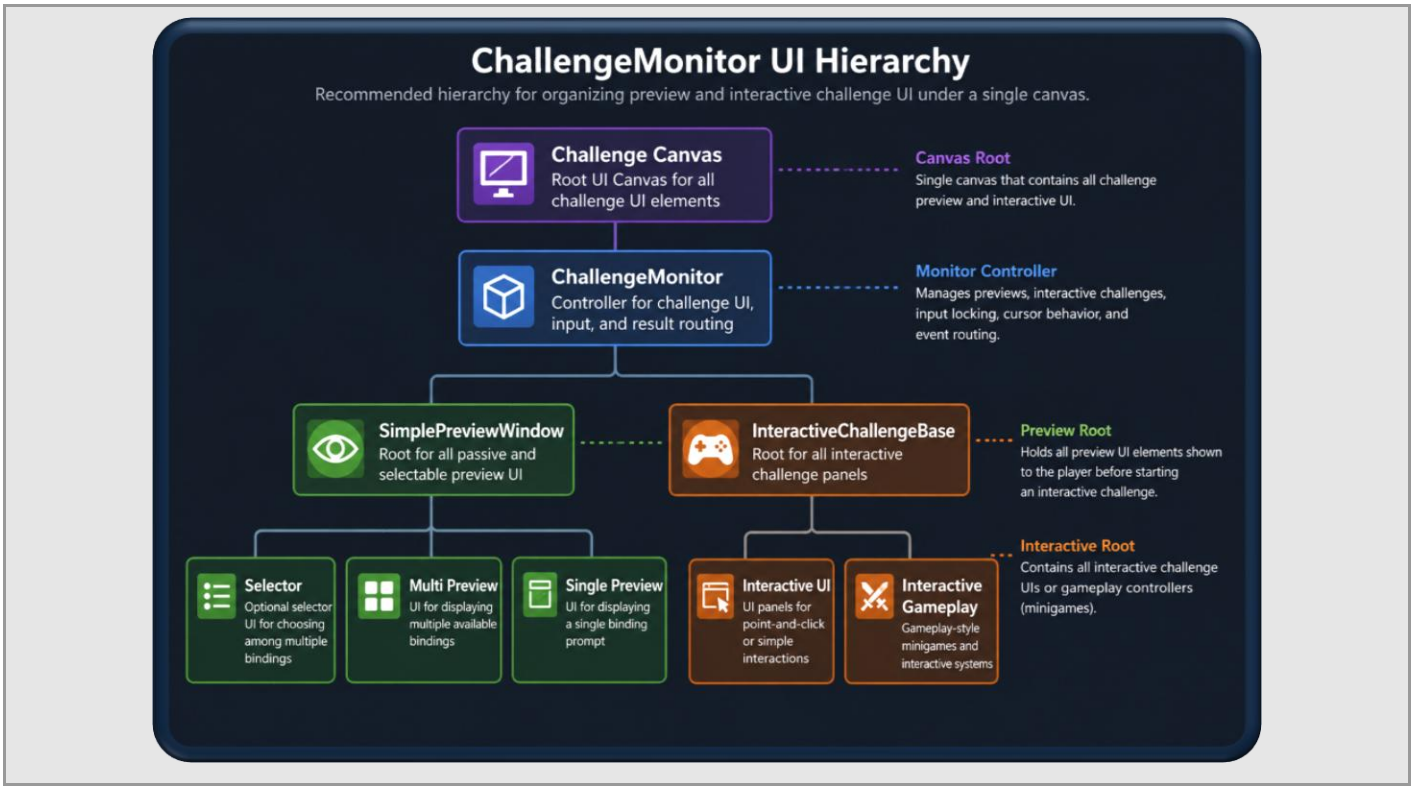
ChallengeMonitor is the UI bridge for the entire UCS player-facing pipeline. It inherits from CharacterMonitor and listens for preview and interactive challenge events from the attached character.



Responsibilities

1. Registers and unregisters events against the active character.
2. Receives OnChallengePreviewRequest and OnInteractiveChallengeRequest events.
3. Manages visibility of all registered Previews and Interactive Challenge UI components.
4. Optionally disables character gameplay input based on configured settings.
5. Optionally locks or confines the cursor based on configured settings.
6. Forwards the challenge result data back to ChallengeActor.

Field	What It Does	Recommended Use / Notes
Disable Character Input	When true, ChallengeMonitor can disable gameplay input while a preview or challenge owns interaction.	Use true for fixed UI panels, keypads, menus, and cursor-driven minigames. Use false only when gameplay movement should remain active.
Previews	Array of ChallengePreviewObject instances.	Each entry is an authored preview style. ActiveSelector chooses which preview style is currently used.
Challenges	Array of InteractiveChallengeBase instances.	Each entry is a custom interactive UI or gameplay challenge that can be started by ChallengeName matching.



Preview Flow

1. ChallengeActor sends OnChallengePreviewRequest.
2. ChallengeMonitor verifies no interactive challenge is already active.
3. ChallengeMonitor selects the active ChallengePreviewObject from the Previews array.
4. If the request is preview-only, selectable callbacks are cleared.
5. If selectable, the preview object receives the current binding list and OnSelect callback.
6. The active preview object displays the prompt or selector.
7. Gameplay input is disabled only when required by the preview style.

Interactive Challenge Flow

1. ChallengeActor sends OnInteractiveChallengeRequest.
2. ChallengeMonitor finds a matching InteractiveChallengeBase in the Challenges array.
3. The default lookup compares Challenge.ChallengeName to the UI GameObject name.
4. If no match exists, ChallengeMonitor cancels the attempt.
5. If a match exists, ChallengeMonitor hides active previews and starts the challenge.
6. The challenge returns success, failure, reset, or cancel events.
7. ChallengeMonitor forwards the final result back to ChallengeActor.

UI Matching Rule

IMPORTANT: By default, the Challenge asset's ChallengeName should match the InteractiveChallengeBase GameObject name exactly.

Option	Meaning
Challenge Name = Password	Starts the UI GameObject named Password.
Challenge Name = Keypad	Starts the UI GameObject named Keypad.
Challenge Name = Lockpick	Starts the UI GameObject named Lockpick.

WARNING: If your challenge cancels immediately, verify that the ChallengeName and UI GameObject name match character-for-character.

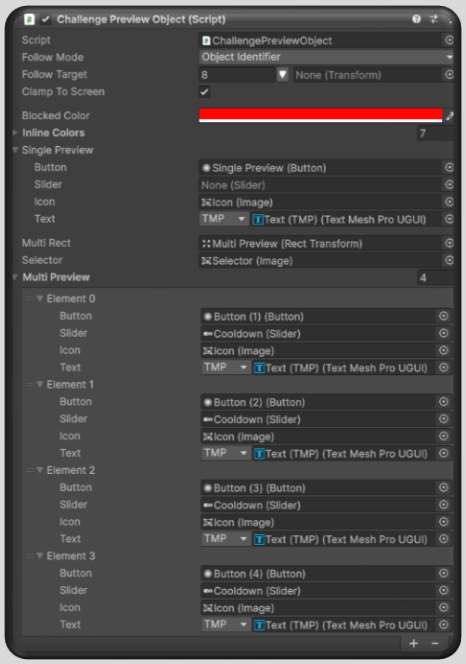
Cursor and Input Ownership

1. SetCursorLocked changes the active Opsive input backend cursor lock behavior.
2. SetCursorConfined changes Unity Cursor visibility and lock state for cursor-style UI interaction.
3. InteractiveChallengeBase can request cursor visibility or cursor-follow elements.
4. ChallengePreviewObject disables input when using fixed UI and leaves input available for world-follow preview styles.

ChallengePreviewObject

ChallengePreviewObject renders a Challenge.PreviewRequest. It is responsible for the visible preview slots, icons, labels, world-follow behavior, selector cursor, and blocked/bypassed interaction states.

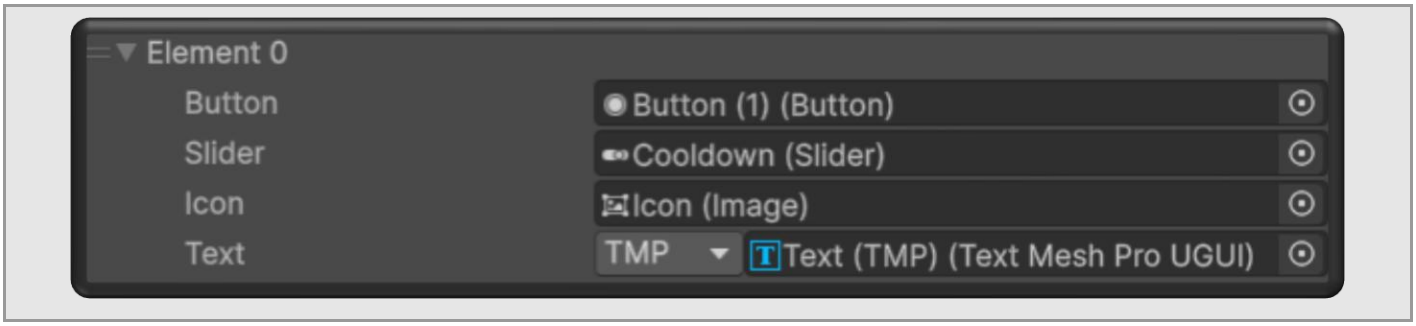
Field	What It Does	Recommended Use / Notes
Follow Mode	Determines how the preview anchors itself.	None behaves as fixed UI. GameObject follows the subject transform. ObjectIdentifier follows a specific child/parent ID.
Follow Target	Optional IDObject used when Follow Mode is ObjectIdentifier.	Use when the preview should appear above a specific child transform, such as a terminal screen or door handle.
Clamp To Screen	Keeps the preview inside the canvas bounds.	Recommended for world-follow previews.
Blocked Pressed Color	Pressed color override used when clicking a blocked option.	Use red or warning colors for denied feedback.
Inline Ingredient Colors	Color set used when rendering ingredient labels inline.	Use readable dark colors that work on your UI background.
Main Rect	Root RectTransform for this preview style.	Defaults to this transform if empty.
Selector	Optional cursor image for world-follow selectable UI.	Useful when gameplay cursor is hidden but UI selection still needs visual feedback.
Multi Preview	Array of authored visual slots for multiple bindings.	Use for ResolveAny, ResolveAll, NextFound, or any object with multiple options.
Single Preview	Authored visual slot for one binding.	Use for simple one-option prompts.



NOTE: Do not hide unused multi-preview buttons by destroying them at runtime. The preview system clears unused slots and makes them non-interactable while preserving authored references and callbacks.

Preview Slot Fields

Field	What It Does	Recommended Use / Notes
Button	Selectable/button component for this slot.	Required for selectable previews. Keep OnClick wired to SelectIndex or the appropriate slot selection method.
Slider	Optional Slider used to display selection cooldowns.	This is primarily used when FollowMode is set to None – the slider acts as a visual indicator for how long you have before character gameplay input is locked.
Icon	Image used to display the binding or ingredient icon.	Optional but recommended for readability.
Text	Opsive Shared.UI.Text label used for preview text.	Can be backed by TextMeshPro for rich text and inline sprites.



Single vs Multi Preview

1. **Single Preview:** Best for one challenge on one object, such as "Requires Keycard".
2. **Multi Preview:** Best when the same object offers several selectable bindings, this only becomes relevant when your challenge group's resolver mode is set to either ResolveAllBindings or ResolveAnyBinding.

Follow Modes

Option	Meaning
None	Fixed UI. Best for centered prompts, selection menus, and UI panels. Usually disables gameplay input when selectable.
GameObject	Follows the subject transform. Best for world labels over doors, terminals, pickups, or switches.
ObjectIdentifier	Follows a specific transform found by ID. Best when the root object is large but the prompt should appear over a screen, handle, keypad, or socket.

PRO TIP: Use Follow Mode None for menus. Use GameObject or ObjectIdentifier for diegetic world prompts that should not fully interrupt movement.

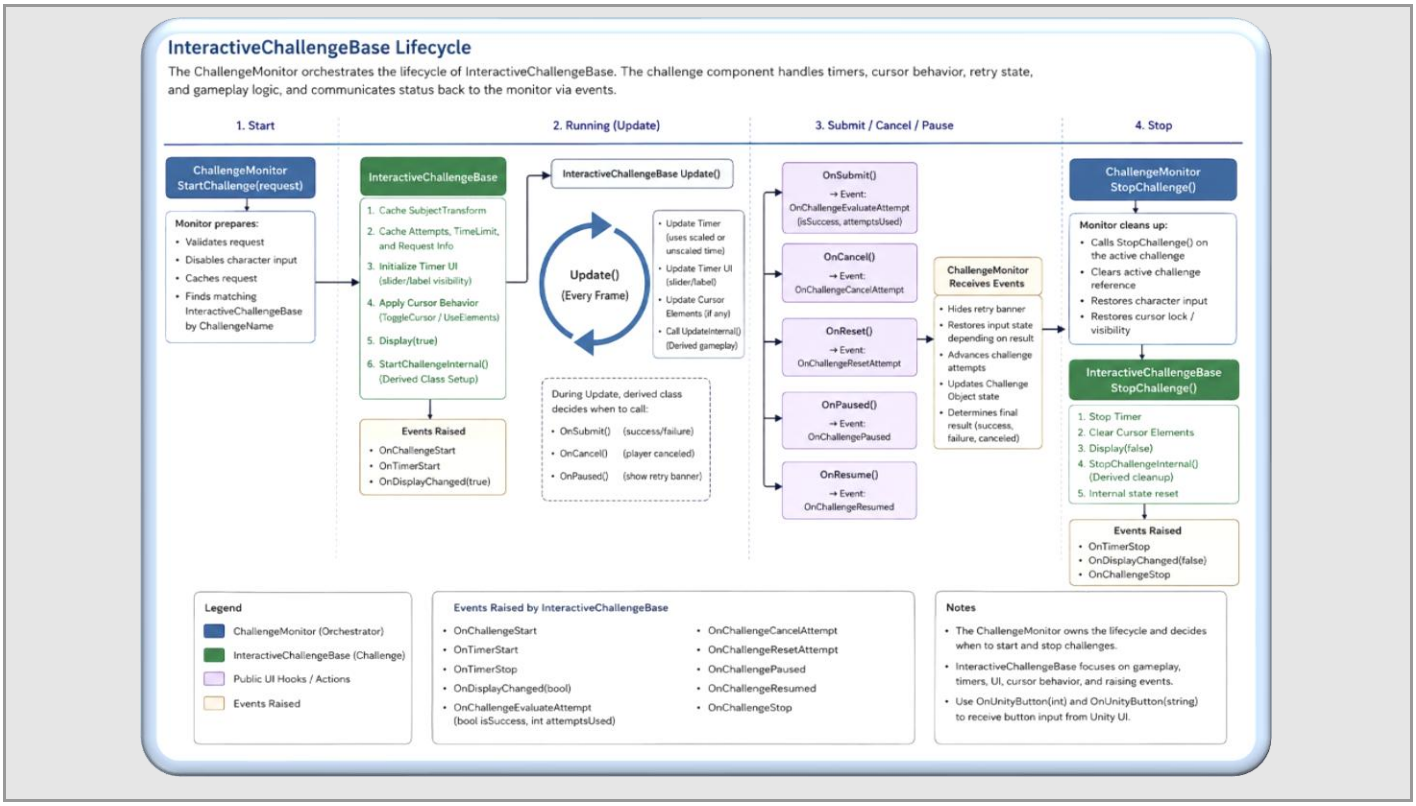
InteractiveChallengeBase

InteractiveChallengeBase is the foundation for challenge UIs and minigames. It handles lifecycle, timer state, retry state, cursor behavior, and event dispatch so derived classes can focus on gameplay logic.

Field	What It Does	Recommended Use / Notes
Timer Slider	Slider showing normalized remaining time.	Optional. Use for timed challenges.
Timer Label	Text label showing remaining time.	Optional. Useful for mm:ss countdowns.
Retry Banner	RectTransform shown when retry/pause/reset state is active.	Use for "Try Again" or "Failed" overlays.
Hide Timer When Unlimited	Hides timer UI when TimeLimit <= 0.	Recommended true to avoid showing useless timer UI.
Show Minutes Seconds	Formats timer as mm:ss.	Use true for longer timers; false for short precise seconds.
Use Unscaled Time	Countdown uses Time.unscaledDeltaTime.	Recommended for UI challenges so timescale changes do not break the timer.
Auto Submit	Allows derived challenge to submit automatically when win condition is met.	Useful for password length match, Simon Says sequence completion, or alignment solved.
Toggle Cursor	Unlocks/shows cursor behavior while challenge is active.	Use for mouse-driven UI panels.
Use Elements	Confines cursor and updates custom cursor-follow elements.	Use for ghost cursor, delayed cursor, or UI elements that trail the mouse.
Cursor Elements	RectTransforms that follow cursor samples with delay/lerp behavior.	Use for challenge effects that require delayed or smoothed cursor visuals.

Base Lifecycle

1. ChallengeMonitor calls StartChallenge(request).
2. The base caches subject transform, attempts, time limit, and request data.
3. Timer UI is initialized.
4. Cursor behavior is applied.
5. Display(true) is called.
6. StartChallengeInternal runs in the derived class.
7. Update handles timers and calls UpdateInternal.
8. The player submits, cancels, pauses, resumes, or fails.
9. ChallengeMonitor receives the result event.
10. ChallengeMonitor calls StopChallenge and restores cursor/input behavior.



Public UI Hooks

Option	Meaning
OnSubmit	Submits the current challenge result.
OnCancel	Cancels the current attempt and returns a canceled result.
OnReset	Requests a retry/reset if attempts remain.
OnResume	Resumes after retry/pause state.
OnPaused	Displays the retry banner when appropriate.
OnUnityButton(int)	Receives integer values from Unity UI buttons.
OnUnityButton(string)	Receives string commands from Unity UI buttons.

PRO TIP: Wire simple keypad and menu buttons directly to OnUnityButton, OnSubmit, OnCancel, and OnReset. You do not need a custom input framework for simple UI panels.

Writing a New Interactive Challenge

1. Create a new Class that derives from InteractiveChallengeBase.
2. Add serialized UI references for your minigame.
3. Override InitializeInternal if you need to cache references.
4. Override StartChallengeInternal to initialize runtime state from the Challenge asset.
5. Override UpdateInternal for per-frame gameplay.
6. Override SubmitInternal to return success or failure.
7. Override ResetInternal if retries need special cleanup.
8. Override StopChallengeInternal to release state, animations, or temporary visuals.
9. Add the component to a UI GameObject assigned in ChallengeMonitor.Challenges.
10. Name the UI GameObject to match Challenge.ChallengeName.

InteractiveGameplayBase

InteractiveGameplayBase is a higher-level base for gameplay-driven interactive challenges. Instead of hardcoding every UI button, it evaluates InteractiveActionGroups using the character input backend and executes InteractiveActions against InteractiveElements.

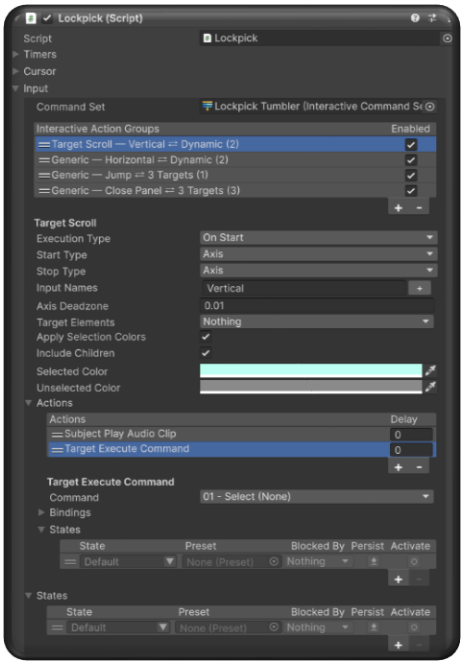
Field	What It Does	Recommended Use / Notes
Interactive Elements	Cached child InteractiveElement components.	These are the UI/gameplay targets that actions operate on.
Command Set	InteractiveCommandSet defining the available command vocabulary.	Use one set per minigame style to keep commands consistent.
Groups	SerializeReference array of InteractiveActionGroup instances.	Each group handles input, lifecycle, and actions.
Active Element Index	Current selected InteractiveElement.	Used by TargetSwitchBase group types to cycle between different child InteractiveElements.

NOTE: InteractiveGameplayBase is best for challenges that behave like small gameplay systems: rotating tumblers, selecting targets, timed switches, Simon Says panels, lockpick pins, or cursor-driven minigames.

Interactive Action Groups

InteractiveActionGroups are reusable input/lifecycle containers. Each group decides when it starts, when it stops, and when its actions execute.

Field	What It Does	Recommended Use / Notes
Enabled	Can this group run?	Disable groups temporarily without deleting them.
Execution Type	When actions execute.	Use OnStart for one-shot commands, WhileActive for continuous motion, OnStop for release behavior.
Start Type	How the group starts.	Automatic, Manual, ButtonDown, ButtonDownContinuous, DoublePress, LongPress, Tap, or Axis.
Stop Type	How the group stops.	Automatic, Manual, ButtonUp, ButtonDown, ButtonToggle, LongPress, or Axis.
Input Names	Opsive input names used by start/stop checks.	Match your project input names exactly.
Wait For Double Press/Tap Timeout	Delays ButtonDown behavior so double press/tap can be distinguished.	Use only when a binding must share the same button with tap/double press behavior.
Long Press Duration	Duration required for LongPress.	Tune per minigame.
Wait For Long Press Release	Requires release after a long press before triggering.	Useful to prevent accidental immediate execution.
Axis Deadzone	Threshold used for Axis start/stop types.	Increase if analog noise causes false starts.
Target Elements	Elements this group operates on.	Leave empty for monitor/global actions or assign one or more UI targets.
Actions	Actions executed by the group.	Order matters. Keep each group focused.



Default Group Types

A few different InteractiveActionGroups have been included out of the box:

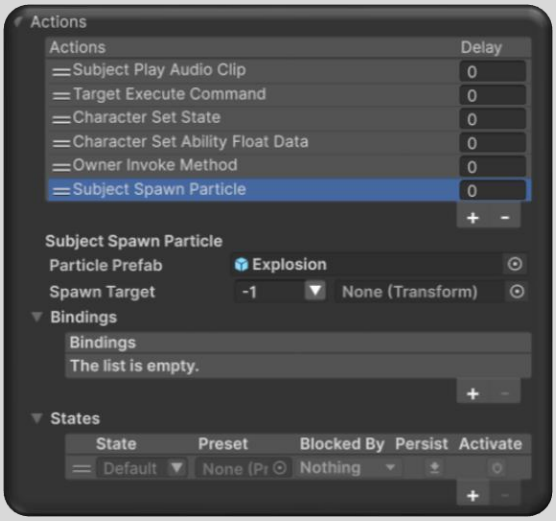
Option	Meaning
GenericInteractiveGroup	General-purpose group for executing actions from input or lifecycle events.
TargetPrevious	Cycles the active element index backward.
TargetNext	Cycles the active element index forward.
TargetScroll	Cycles the active element index forward and backward based on Axis input values
TargetSpecific	Maps specific input names to specific element indices.

PRO TIP: Prefer several small action groups over one giant group. Small groups are easier to debug, disable, and reuse.

Interactive Actions

InteractiveActions are modular effects executed by InteractiveActionGroups. They are intended to be reusable building blocks used by InteractiveGameplayBase-derived challenges.

Option	Meaning
CharacterSetAbilityFloatData	Sets the character ability float data value, optionally with damping.
SetState	Activates or changes a named State for feedback or behavior changes.
OwnerInvokeMethod	Invokes a selected public method on the owner challenge.
ConditionalInteractiveAction	Runs one set of actions if conditions pass and another if they fail.
DebugLog	Writes a debug message for testing action flow.
MonitorInvokeMethod	Invokes a method on the ChallengeMonitor.
PlayAudioClipSet	Plays a random clip from an AudioClipSet.
SpawnParticle	Spawns a particle prefab, optionally at an IDObject target.
SubjectSetActive	Enables, disables, or toggles a GameObject subject.
TargetExecuteCommand	Sends a command payload to an InteractiveElement.
TargetFlashColor	Flashes target graphics using a temporary color.



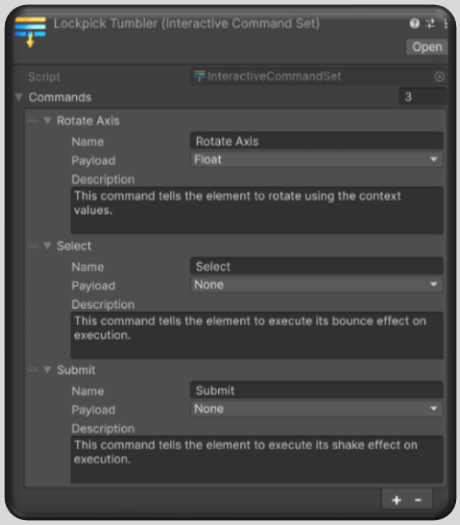
Field	What It Does	Recommended Use / Notes
Delay	Optional delay before an action executes.	Use for staggered effects, audio timing, animation timing, or delayed feedback.

NOTE: Actions should be small and composable. If one action is becoming a full minigame, move that logic into the owner InteractiveChallengeBase and call it through an action.

InteractiveCommandSet

InteractiveCommandSet acts as a common vocabulary between all child InteractiveElements. This avoids hardcoding button-specific behavior directly into action groups, and allows designers to use their own custom solutions for producing effects.

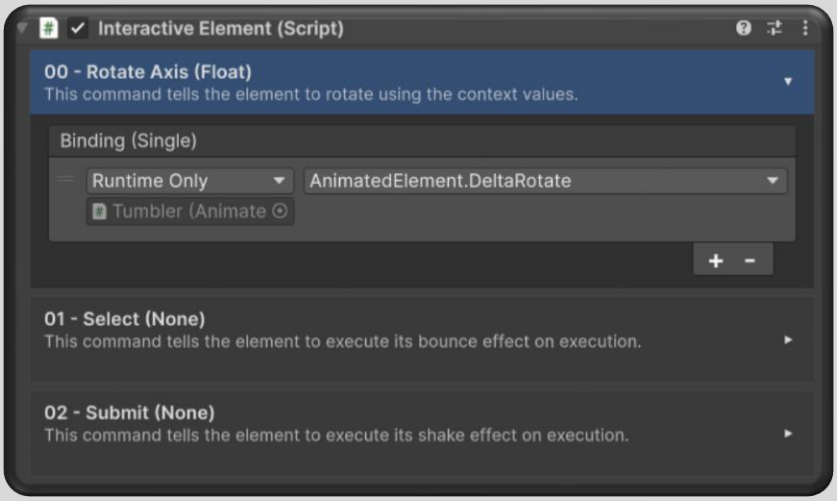
Field	What It Does	Recommended Use / Notes
Command Name	Human-readable command name shown in editor dropdowns.	Use clear names like RotateLeft, Confirm, Flash, ToggleLight, SetCorrect.
Payload Type	None, Float, Bool, or Context.	Choose based on what the receiving element needs.
Description	Optional explanation of the command.	Use this to help designers understand what the command is intended to do.
Command ID	Stable SerializableGuid for the command.	Keeps references stable even if the display name changes.



InteractiveElement

This component is designed to take common commands from an InteractiveAction and translate that into hardcoded events for this GameObject only. This bridge component allows designers the flexibility to implement their own specialized controllers for whatever 2D element is being manipulated.

Field	What It Does	Recommended Use / Notes
Command Bindings	List of command IDs mapped to UnityEvents.	Each element decides how it responds to each command.
OnInvoke	UnityEvent for commands without payload.	Use for simple triggers.
OnInvokeFloat	UnityEvent<float> for numeric commands.	Use for rotation, progress, scale, sliders, or numeric effects.
OnInvokeBool	UnityEvent<bool> for boolean commands.	Use for toggles, correct/incorrect flags, active states.
OnInvokeContext	UnityEvent<InteractiveCallbackContext> for rich payloads.	Use when the element needs owner, monitor, subject, target, or additional context.

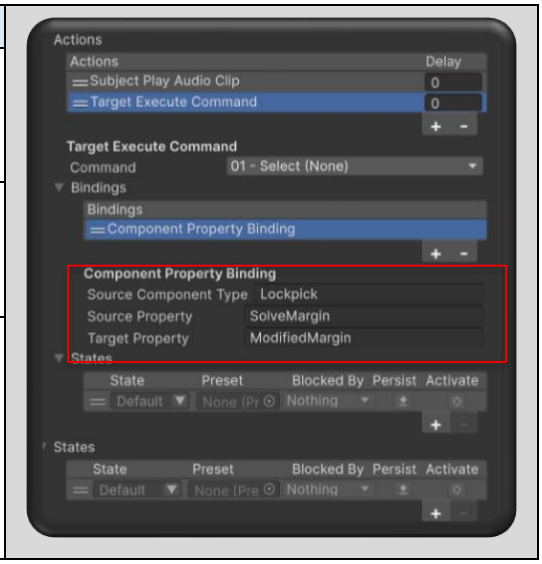


PRO TIP: Command IDs make command references resilient. Designers can rename commands without breaking every action that points to the command.

Component Property Bindings

ComponentPropertyBinding is used when a StateObject/action property should be driven from a component property on the bound GameObject. This is useful for minigames where authored actions should read runtime values from the owning challenge.

Field	What It Does	Recommended Use / Notes
Source Component Type	Type name of the component on the bound GameObject.	Examples: Lockpick, Password, MyNamespace.MyComponent
Source Property	Property name to read from the source component.	Example: RotationStep, IsCached, CurrentIndex.
Target Property	Property on this StateObject/action to write.	Example: BoundValue, Toggled, FloatData.



NOTE: Use bindings for values that designers should author once but that need to follow runtime state. Avoid bindings for one-off logic that is clearer as a direct method call.

Experimental

ChallengeObject

ChallengeObject is an optional world-side persistence helper. Use it when a Persistent challenge should remember outcome, attempts, or resolved state after save/load.

Field	What It Does	Recommended Use / Notes
ObjectIdentifier ID	The ID used to identify the world object.	Should match the Challenge TargetID.
Challenge reference/key	Identifies which challenge data this object is tracking.	Use stable challenge references when saving.
Resolved state	Whether the challenge has been solved, failed, or remains unresolved.	Restored into ChallengeActor when the object is loaded.
Attempts	Remaining or recorded attempts.	Important for OnReset persistent challenges.
Outcome	Last ChallengeOutcome.	None, Success, or Failure.

Automatic Abilities

Automatic abilities require special attention because they will attempt to start every frame while their start conditions remain true. A normal ButtonDown interaction is a single input event; an Automatic interaction is a continuous pressure against the challenge gate.

Why Automatic Abilities Are Different

1. An Automatic ability can repeatedly report that it can start while the character remains near or aimed at the Detected Object.
2. If ChallengeActor simply stops, the automatic ability may immediately try again on the next frame.
3. UCS uses preview refreshes, grace intervals, active latching, and selected binding state to prevent flickering and repeated start/stop loops.
4. Automatic abilities must remain gated until the player leaves the target, the binding is bypassed, or the selected challenge can safely evaluate.

Recommended Automatic Setup

1. Use Automatic only when the original ability truly needs continuous start evaluation.
2. Keep Refresh Interval reasonable so cooldown previews update but do not spam UI rebuilds.
3. Use Grace Interval to avoid flickering when detection briefly fails.
4. Test target enter, hover, leave, success, failure, and cooldown while standing still and while moving through the trigger.
5. Avoid complex ResolveAllBindings automatic setups until single-binding automatic behavior is stable.

WARNING: When debugging automatic abilities, watch for loops: ChallengeActor starts, stops the intercepted ability, ChallengeActor stops, the intercepted ability immediately tries again. If this occurs, the automatic gate is not remaining active long enough or the selected binding state is not being preserved; try increasing the Grace Interval float value, since this may indicate that the DetectedObject is being gained and then lost repeatedly – this happens most often when intercepting automatic abilities using Trigger Zones as their detection method. If the issue persists, make sure any animations you are playing do not have root motion enabled, since this can physically move the character in and out of detection range.

- It should be noted here that a lot of the issues you may encounter with this interception method are inherent to the DetectObjectAbilityBase itself – try to tweak the settings there so you are not polling the detection method every frame.
- In addition to this, if you are using an automatic Interact ability to open a locked gate (requires a inventory key), make sure the associated AnimatedInteractable Component attached to the DetectedObject is set to Single Interaction to prevent repeated Interact attempts. Play around with the Abilities in question and how they behave before attempting to gate them using ChallengeActor.

Troubleshooting

The challenge never starts

1. Confirm ChallengeActor is on the character.
2. Confirm ChallengeActor uses SynchronousStarter.
3. Confirm the desired intercepted ability is selected inside SynchronousStarter.
4. Confirm the target object has an ObjectIdentifier that matches the Challenge's TargetID.
5. Confirm the detected collider is in the hierarchy that can resolve the ObjectIdentifier.
6. Confirm ability ordering has not invalidated the SynchronousStarter cache – this can be resolved by opening the ChallengeActor ability and allowing the Inspector to redraw the SynchronousStarter.

The preview appears but the original ability never resumes

1. Check whether the selected binding is blocked instead of bypassed.
2. Check Runs/Reset settings on the Challenge asset.
3. Check whether FoundNextBinding is waiting for another binding to be solved.
4. Check whether the intercepted ability can still start after the challenge completes – you can confirm this by simply setting the ChallengeActor ability to m_Enabled = false either in the inspector or via StateSystem Preset.

Interactive challenge cancels immediately

1. Check ChallengeMonitor exists and is attached to the character UI.
2. Check ChallengeName exactly matches the InteractiveChallengeBase GameObject name.
3. Check the UI GameObject is active or can be activated.
4. Check the component is included in the ChallengeMonitor's Challenges array.
5. Ensure no other interactive challenge is already active.

Inline icons do not display

1. Generate TMP sprite assets from the Add-Ons Manager.
2. Assign the generated sprite asset as the TMP default sprite asset.
3. Verify the TextMeshPro component supports rich text and sprites.
4. Verify the item icons exist on the CharacterItem or ItemType prefabs inside your ItemCollection.

Automatic abilities loop or flicker

1. Increase Grace Interval slightly.
2. Verify ChallengeActor remains active while the automatic ability is still targetable.
3. Avoid repeatedly clearing pending bindings during automatic hover.
4. Test with one binding before testing multi-binding automatic interactions.
5. Make sure lost target detection is not firing too aggressively.
6. Alter the settings on the automatic ability's detection method – try increasing the cast frame interval, shortening the cast distance, etc.